

33. Price Tracking Service (CamelCamelCamel / Keepa-style)

Interviewer: Design a price-tracking service like CamelCamelCamel or Keepa. Users add a product (say, an Amazon ASIN) and a **target price**. We track the product's price over time across retailers, show a **price-history chart**, and **notify the user the instant the price drops to or below their target**. We need to do this for **millions of products** and millions of user-defined alerts. Design it.

Candidate: This is a fun one because it looks like a scraping problem on the surface, but the two real challenges are underneath: (1) we can't poll millions of products at high frequency forever — there's a **crawl budget** — so we need smart prioritization, and (2) when a price *does* change, we can't scan every alert in the system to see who cares — that's a **matching-at-scale problem**, basically a reverse index. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions: 1. Are we scraping retailer pages ourselves, or do we have official price-feed APIs for some of them? (Changes whether "crawl" means HTTP scraping or polling a partner API.) 2. How fresh does a price need to be? If Amazon drops a price at 2:00pm, is it OK if we notice it at 2:05pm? 2 hours? 3. One alert per user per product, or can a user have multiple thresholds (e.g., "notify at \$80, and again at \$60")? 4. Do we need the **full price history forever**, or just enough to draw "last 6 months"?

Interviewer: Mostly scraping, a few official feeds. Freshness: a few minutes for popular products is great, hours is acceptable for obscure ones — it does **not** need to be real-time for everything. One active threshold per user per product for simplicity. Keep history indefinitely — "all-time low" is a headline feature.

Candidate: Good, that "freshness can vary by popularity" answer is the key constraint that shapes the whole crawler design. Requirements:

Functional - Track price-over-time for millions of products across one or more retailers. - User adds a **watch**: {userId, productId, targetPrice}. - The instant a newly observed price for a product is $\leq$ targetPrice for some watch, **notify that user** (email/push). - Serve a **price-history chart** (and derived stats: all-time low/high, average) per product.

Non-functional - **Crawl efficiency at scale:** millions of products, can't hit them all at a fixed high rate — budget-constrained (retailer rate limits, our own infra cost). - **Alert-matching must scale with the number of watchers of the product that changed, not the number of watches in the whole system.** A price tick on one item must never trigger an O(all watches) scan. - **Write-heavy time-series storage:** every price observation is a write; history grows forever and must support fast "give me the last N months" reads. - **Notification latency:** minutes, not hours, once a price is actually observed. - **No**

missed alerts, no duplicate spam (notify once per crossing, not on every subsequent tick while the price stays low).

2. Estimation — the two numbers that define the design

Catalog: ~500M tracked product-retailer pairs (e.g. Keepa-scale: every Amazon ASIN x region).

Users: ~20M active users, average 5 watches each → ~100M active watches.

CRAWL BUDGET:

A fleet of scraper workers, each issuing ~2 req/sec (politeness + retailer rate limits), 5,000 workers concurrently:

throughput $\approx 5,000 * 2 = 10,000$ checks/sec ≈ 864 M checks/day

If we tried to poll ALL 500M products every 5 minutes (naive, uniform):

$500,000,000 / 300s = \sim 1.67$ M checks/sec required

...that's ~167x our actual crawl budget. Uniform high-frequency polling of the full catalog is impossible before we even discuss retailer bans.

⇒ We must SPEND the 10k/s budget non-uniformly: poll popular / volatile / actively-watched products often, poll the long tail rarely.

ALERT-MATCHING BUDGET (per price update):

A product update must find "which watchers of THIS product are now satisfied?" If a product has 50 watchers, trivial. If it's an iPhone with 3M watchers and the price just dropped \$50, we need to find exactly which subset crossed their threshold — WITHOUT re-scanning irrelevant watches on unrelated products, and without re-scanning the full 3M every tick.

STORAGE:

$864\text{M checks/day} * \sim 100 \text{ bytes/point (product_id, ts, price, retailer)} =$
 $\sim 86 \text{ GB/day of raw price ticks if we stored every poll. We'll dedupe}$
(only write a point when price actually CHANGES) to cut this drastically.

Candidate: So the two hard problems are exactly the ones in the prompt: **crawl prioritization** (spend a fixed budget where it matters) and **alert matching** (a targeted lookup, not a scan). Everything else — storage, charts, notifications — is comparatively standard once those two are solved.

3. A simple V1

Candidate: Let me state the naive version, purely to agree on where it breaks.

[Cron: for each product in DB, every 10 min]



[Scraper hits retailer page] → [INSERT price_point(product_id, ts, price)]



[For every watch on this product: if price ≤ target → send email]

This "works" for a demo with 1,000 products and 10,000 watches. It falls over at our scale for two independent reasons: 1. **The cron polls everything at the same frequency** — see the 1.67M/s math above. We need tiered, priority-driven scheduling instead of a flat loop. 2. **"For every watch on this product"** is fine *if* it's indexed by product — that part is actually OK at moderate watcher counts. The real danger (which the interviewer will push on) is if the naive implementation instead does "for every watch **in the system**, check if `watch.product_id == this_product`" — i.e., a full table scan per price tick. We'll come back to this in the deep-dives; for now, assume we index watches by `product_id` from day one.

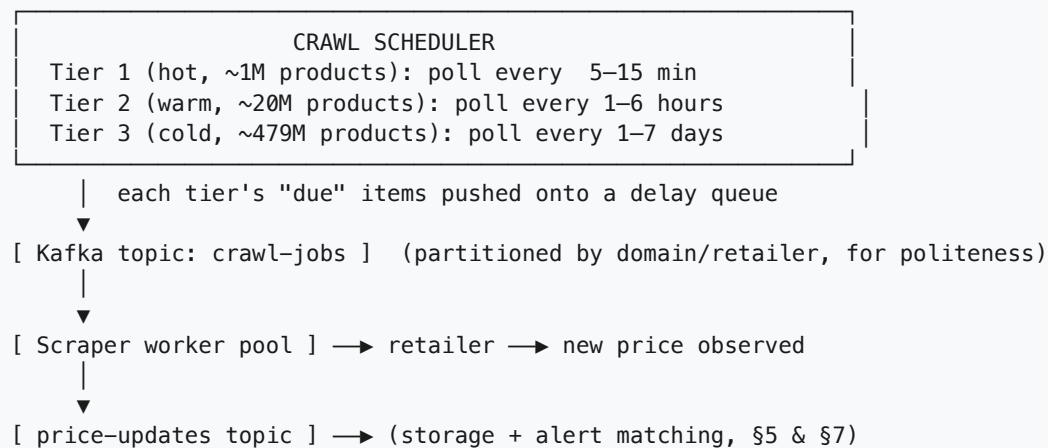
4. Scaling the crawler — tiered, priority-driven polling

Interviewer: Right, so how do you actually decide what to crawl, and how often?

Candidate: Treat crawling as a **priority queue problem**, not a fixed-interval cron. Every product gets a **poll priority score**, and a scheduler continuously pulls the highest-priority due items off a queue, spending the fixed 10k/s budget on whichever products matter most *right now*.

Priority signal per product (recomputed periodically):

- # of active watches on it (more watchers → higher priority)
- historical volatility (price that swings a lot → check more often)
- recency of last change (just changed → worth rechecking soon)
- "closeness to a threshold" (a watch with target \$2 above current price is one small dip away from firing → bump priority)



- **A delay/priority queue** (e.g., a Redis sorted set keyed by `next_poll_time`, or a scheduled-message feature on the queue) holds "when is this product next due." A scheduler pops everything with `next_poll_time <= now`, in priority order, up to the budget for that tick, and enqueues crawl jobs.
- **Tiers re-balance automatically:** a product with zero watches drifts to Tier 3; the moment a user adds a watch on it, its priority jumps and it's promoted to Tier 1/2 on the next scheduling pass. This means **"a user is actively watching this" itself becomes crawl signal** — we never make a user wait days for the very item they just asked us to track.
- **Partitioning by retailer domain** in the queue lets us respect **per-retailer rate limits** independently (Amazon's limit shouldn't starve Walmart's crawl budget), and lets us swap in an

official price-feed API for a retailer without touching the scheduler.

5. Price-history storage — a time-series problem

Interviewer: Every price check produces a data point. How do you store years of price history for 500M products and still render a fast chart?

Candidate: Two things make this a textbook time-series workload: writes are **append-only, ordered by time, keyed by product**, and reads are almost always **"give me a time range for one product,"** rarely random point lookups. That's exactly what a time-series store is built for.

```
Table: price_history
  (product_id, retailer, ts, price)  partition/shard key = product_id
                                   clustered/sorted by ts within each product
```

Write: only when price CHANGES (dedupe at the scraper) -> cuts ~90%+ of writes, since most polls return the same price as last time.

Read: "chart for product X, last 6 months" = one range scan on one partition.

- **Dedupe-on-write:** the scraper compares the new price to the last stored price for that product; if unchanged, it just bumps a `last_checked_at` field and writes nothing to history. This turns 864M polls/day into a much smaller number of actual history rows — most products' prices don't move most of the time.
- **Downsampling for old data:** keep raw (minute/hour-level) resolution for recent months, roll older data up into daily min/max/avg buckets. A 3-year-old chart doesn't need minute-granularity points — nobody can see the difference on a chart, and it keeps storage bounded per product instead of growing unboundedly.
- **Why a time-series-oriented store (Cassandra / a TSDB like TimescaleDB) and not a plain relational table:** we need **fast writes partitioned by key + time-ordered reads by key**, at a scale (billions of rows) where a single Postgres table's B-tree index and vacuum overhead become the bottleneck. Cassandra (or DynamoDB with a similar key design) gives us `product_id` as the partition key and `ts` as the clustering key for free — a range query for "this product, this time window" hits one partition, one sequential read. A relational table would need the same conceptually, but at 500M products x years of history, its single-node write throughput and index maintenance cost lose to a store designed to partition by key from the start.

6. API + data model

POST /watches	{productId, targetPrice}	-> create/update a watch
GET /products/{id}/history?range=6m		-> chart data
GET /products/{id}		-> current price, all-time low/high
DELETE /watches/{id}		-> stop watching

```
watches table (relational – Postgres):
  watch_id | user_id | product_id | target_price | status | created_at
  index on (product_id, target_price)  <-- THE index that makes alert
                                         matching a lookup, not a scan

products table (relational):
  product_id | name | retailer | url | current_price | last_checked_at

price_history (time-series store – Cassandra/TimescaleDB):
  partition key: product_id   clustering key: ts   value: price
```

Why relational for watches and products, but time-series for price_history? Watches and products are **low-volume, highly relational** (joins to users, need transactional updates, secondary indexes for the matching engine) — a great fit for Postgres. Price history is **enormous, append-only, and queried almost exclusively by (product_id, time range)** — a relational B-tree buys us nothing there and costs us write throughput; a partitioned time-series store is the right tool. Using one database for everything would mean either crippling the watches table's flexibility (to fit a TSDB) or crippling the history table's write throughput (to fit Postgres) — split by access pattern instead.

7. The alert-matching engine — the reverse-index problem

Interviewer: Here's the core of it. A price update comes in for product X. How do you find every user whose alert just became true — fast, and without scanning things that don't matter?

Candidate: The key realization: this is a **reverse lookup, not a scan** — "given a value (the new price), find all watches on *this product* whose threshold the value now satisfies." Two design points make it fast:

1. **Index watches by product_id first.** A price event for product X should only ever touch the watches *on X* — never the other 100M watches on other products. This is why the (product_id, target_price) index in §6 matters: it's not just a schema nicety, it's what turns the operation from O(all watches) into O(watches on this one product).
2. **Within one product's watches, keep thresholds in a sorted structure**, not an unsorted list, so "which of these thresholds does the new price satisfy" is a **range query**, not a linear scan of that product's (possibly millions of) watchers. A **Redis sorted set** per product — ZADD alerts:{productId} targetPrice userId — is a natural fit: $O(\log n)$ insert, and a range query (ZRANGEBYSCORE alerts:{productId} newPrice +inf) returns exactly the watchers whose targetPrice $\geq$ newPrice (i.e., the new price is at or below their target) in one shot.

```

[ price-updates topic ] price(X) = $180 (was $210)
    |
    v
[ Alert-matching worker ]
    |
    ZRANGEBYSCORE alerts:X 180 +inf
    |   -> {userId: 501, target: 200}, {userId: 892, target: 180}, ...
    |       (every watcher whose target is >= the new price -> condition met)
    |
    v
[ ZREM those members from alerts:X ] <-- stop re-matching them every tick
    |                                   while price stays low (see §8/§13)
    |
    v
[ enqueue one notification per matched userId ]

```

We'll dig into exactly how this holds up when X has millions of watchers in **Deep-dive B** — the range query is right, but "millions of matches at once" still needs care on the fan-out side.

8. Notification fan-out, caching, and charts

Notification fan-out: matched (userId, productId) pairs are pushed onto a **queue** (Kafka, partitioned by userId so per-user ordering/rate-limiting is easy), and a pool of notification workers drains it, calling email/push providers. This is off the alert-matching critical path — matching just needs to identify *who*, not wait on a slow email API per user.

```

[ Alert-matching worker ] → [ Kafka: notifications ] → [ Notify workers ]
                                     |
                                     v
                                [ Email/SES ]      [ Push/APNs/FCM ]

```

Caching for charts: a product's price-history chart is read far more often than it changes (a viral "deal alert" post can drive thousands of chart views for one product in minutes, while the underlying price updates maybe once an hour). Cache the rendered chart-series (e.g., "last 6 months, downsampled to daily points") in Redis/CDN with a short TTL and invalidate/refresh it only when a new price point is actually written — classic write-through-on-change rather than read-through-every-request against the time-series store.

Consistency for the current-price field: products.current_price is a **denormalized cache** of the latest history point, updated in the same write path as the history insert, so reads of "current price" don't have to query the time-series store at all — one more place we trade a tiny bit of duplication for read speed on a hot path.

9. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Price history (time-series)	Cassandra / TimescaleDB , partitioned by product_id, clustered by ts	Append-heavy writes and range-scan-by-key reads at billions-of-rows scale; partitioning by key gives linear write scale-out. <i>Not a single Postgres table</i> — one giant time-

Concern	Choice	Why this, and why not the alternative
		ordered table across 500M products would blow past one node's write throughput and bloat its B-tree/vacuum cost.
Watches / alerts store	PostgreSQL , indexed on (product_id, target_price)	Relational, transactional, needs joins to users and supports the exact index the matching engine needs; volume (100M rows) is trivial for Postgres. <i>Not a pure KV store</i> — we need the composite index for range-style lookups, which a plain KV store doesn't give us for free.
Fast alert-matching index	Redis sorted set per product (ZADD / ZRANGEBYSCORE)	$O(\log n)$ insert, $O(\log n + k)$ range query for "who just crossed" — turns matching into a targeted lookup instead of a scan. <i>Not scanning the Postgres watches table per price tick</i> — even with the right index, doing this synchronously against the primary OLTP store for every one of thousands of price ticks/sec adds needless DB load; Redis is the fast front-line, Postgres stays the durable source of truth.
Crawl scheduling queue	Priority/delay queue (Redis sorted set keyed by next-poll-time) feeding Kafka	Lets the scheduler always pull the highest-priority <i>due</i> items under a fixed budget; Kafka partitioned by retailer domain enforces per-retailer politeness independently. <i>Not a flat cron over all products</i> — that's the 1.67M/s math from §2; a fixed interval can't express "poll this one more often because 3M people are watching it."
Crawl jobs / price-update event bus	Kafka	Durable, replayable, partitioned by retailer/product — decouples scraping from storage and alerting, and lets us replay a burst of updates if a downstream consumer lags. <i>Not direct synchronous writes from scraper to DB</i> — a slow or down time-series store would then block/crash the crawler fleet itself.
Notification fan-out queue	Kafka topic, partitioned by userId	Off the matching critical path; per-user partitioning makes rate-limiting/ordering per user natural. <i>Not calling email/push providers inline from the matching worker</i> — a slow provider would stall matching for the <i>next</i> price event.
Chart cache	Redis/CDN, invalidated on new price point	Charts are read far more than prices change; serve hot reads from cache, refresh only on actual writes.

Say-it-out-loud justification: "A time-series store partitioned by product carries the price history because writes are append-only and reads are range-scans by key; Postgres holds the low-volume, relational watch/alert definitions with the index that makes matching a lookup; a Redis sorted set per product turns 'who just crossed their threshold' into an $O(\log n)$ range query instead of a scan; and Kafka decouples crawling, storage, matching, and notification so a slow retailer or a slow email provider never blocks the others."

10. Consistency model — what's strong, what's eventual

- **Strong-ish (read-your-writes):** once a price point is written to history, `current_price` is updated in the same logical step, so a chart/API read never sees a stale current price relative to the latest write we know about.
- **Eventual:** the crawl itself — we only know the price we last polled; a retailer's *actual* live price can be fresher than our last check by minutes to days depending on tier. This is an accepted, explicit trade-off (§1), not a bug.
- **Alert delivery:** "notify within minutes of *us observing* the drop" is the real SLA, not "notify within minutes of the retailer changing the price" — we can't promise the latter without polling everything constantly, which is exactly the budget problem in §2/§12.
- **Notify-once semantics:** removing a matched watch from the Redis sorted set (§7) after firing is itself a consistency choice — we favor "never double-notify while price stays low" over strict re-evaluation on every tick, and re-arm the watch when the price rises back above target (a fresh `ZADD`).

11. Failure modes & graceful degradation

Failure	What happens	Mitigation
Scraper fleet degraded/rate-limited by a retailer	Crawl lag grows, prices go stale	Back off per-domain, shift budget to other retailers; stale-price banner in UI past a threshold age
Redis (alert index) down	Matching can't run	Alerts queue up as price-update events in Kafka (retained); replay through matching once Redis recovers — delayed, not lost
Time-series store degraded	History writes/reads slow or fail	Kafka buffers price-update events; writers retry/drain on recovery; <code>current_price</code> cache in Postgres still serves "latest price" reads
Notification provider (email/push) down	Users don't get notified in time	Kafka retains the notification queue; workers retry with backoff; no alert is silently dropped
Crawl scheduler bug prioritizes wrong tier	Popular products go stale, cold ones over-poll	Priority score recomputed periodically from real signals (watch count, volatility) — self-correcting, plus alerting on "avg staleness by tier"

12. Follow-up curveballs

Interviewer: A retailer changes their page HTML and your scraper starts silently returning garbage prices. How do you catch that before it fires false alerts?

Candidate: Sanity-check every new price against recent history before writing it: reject (or flag for review) a point that's, say, >80% below the trailing 30-day median — a real flash sale looks like a

temporary dip, a broken scraper looks like "\$0.01" or "\$999999." Alert matching should run on *validated* points only, and a spike in rejected points for one retailer domain pages an on-call to check the scraper.

Interviewer: A user wants to watch a product that's currently at \$50 with a \$60 target — i.e., already satisfied. What happens?

Candidate: Fire immediately on watch creation by checking the current cached price against the new target synchronously in the `/watches` write path, rather than waiting for the next crawl. Otherwise the user waits until the next scheduled poll for a condition that's already true.

Interviewer: How do you support "notify me on *any* price drop," not just below a fixed target?

Candidate: Model it as a threshold that's continuously re-armed relative to the last-notified price: on write, compare the new price to `last_notified_price` (or current price if never notified) instead of a fixed target, and update that watermark after each notification. Same matching machinery — the sorted-set value is just recomputed dynamically instead of being a fixed user-set number.

Interviewer: Users want price history *across retailers* on one chart (Amazon vs. Walmart for the "same" product).

Candidate: Keep `retailer` as part of the partition key in `price_history` (as modeled) so each retailer's series is independently stored and crawled; the chart endpoint fetches multiple series by the same logical `product_id` (mapped via a canonical-product table) and overlays them client-side. No change to the matching engine — alerts are still per (user, retailer- specific-or-canonical product, target).

13. Deep-dive: "Why not just poll everything often, and rescan all alerts on every update?"

Interviewer: Simplify this for me. Just poll every product every few minutes so nothing is ever stale, and when a price changes, loop over every alert in the system and check if it matches. Why not?

Candidate: Both halves of that plan individually blow past a budget we can't spend our way out of — one is a **crawl-budget** problem, the other is a **matching-cost** problem, and they're independent failures.

Why not poll every product at a fixed high frequency

The math is in §2: polling all 500M products every 5 minutes needs ~1.67M checks/sec; our realistic crawl fleet gives ~10k/s. That's not "add more servers" — it's ~167x, and most of that budget would be **wasted**, because the overwhelming majority of products' prices don't change in any given 5-minute window. Worse, retailers actively rate-limit or ban aggressive scraping, so "poll harder" isn't even available as a brute-force option — it gets our IPs blocked. The fix is to **spend the fixed budget where**

it has payoff: popular/volatile/ actively-watched products get frequent polls; everything else gets infrequent polls, because nobody's waiting on them anyway (§4).

Why not scan all alerts on every price update

If the matching step, on a price event for product X, does "loop over every watch row in the system, check `watch.product_id == X`," the cost of matching **one product's price tick** scales with the **total number of watches in the whole system** (100M), not with how many people actually watch X. At thousands of price-update events/sec across the catalog, that's a `100M x thousands/sec` cost — it doesn't matter that most of those 100M rows are irrelevant to this particular event, we paid to look at every one of them anyway.

Naive full scan: `price(X) changes -> scan ALL 100M watch rows,`
`filter product_id == X in application code`
cost per event $\sim O(\text{total watches in system})$

Indexed lookup: `price(X) changes -> ZRANGEBYSCORE alerts:X newPrice +inf`
cost per event $\sim O(\log(\text{watchers of X}) + \text{matches})$

Concern	Naive full-scan matching	Indexed threshold structure (per-product sorted set)
Cost per price event	$O(\text{total watches system-wide})$ — same cost whether the product has 5 or 5M watchers, and whether the <i>system</i> has 1M or 1B watches	$O(\log n + k)$, where n = watchers of <i>this</i> product and k = actual matches — cost tracks the event, not the system size
Behavior as the system grows	Gets worse for every event as total watches grow, even on unrelated products	Unaffected by growth elsewhere in the system — perfectly isolated per product
Behavior on a hot product	Same bad scan as any other product (doesn't get worse, but was already bad)	Still fast to <i>find</i> matches (range query); fan-out to notify millions is the real cost here — see Deep-dive B
Implementation cost	"Just SQL / just a loop" — deceptively simple to write, catastrophic to run at scale	One sorted-set (or composite-indexed range query) per product; a bit more design, pays for itself immediately

Rule of thumb: *Never let the cost of handling one event scale with the size of the whole system — it should scale with the size of the thing the event is about.* A price tick on product X should cost $O(\text{watchers of X})$, never $O(\text{all watches everywhere})$. Crawl frequency has the same shape: never spend budget uniformly across the whole catalog when demand (and volatility) is wildly non-uniform.

14. Deep-dive: the millions-of-watchers fan-out problem

Interviewer: A hot product — say a flagship phone — has **3 million watchers** with different target prices, spread from "notify at any drop" to "only if it hits half price." The price drops from \$999 to

\$899. Walk me through finding exactly who to notify, without scanning 3 million rows, and without spamming people repeatedly while the price stays at \$899.

Candidate: This is the sharpest version of the reverse-index problem from §7 — the *lookup* is easy with a sorted structure, but *processing and notifying* a huge match set safely is where the care is needed.

The setup

```
alerts:iPhone16 (Redis ZSET, member = userId, score = targetPrice)

score:  500   750   800   850   880   899   920   950   999  1200 ...
member: u1    u2    u3    u4    u5    u6    u7    u8    u9   u10 ...

price drops 999 -> 899
everyone with score >= 899 now has a satisfied alert:
{u6(899), u7(920), u8(950), u9(999), u10(1200), ...} <-- could be ~1.5M of the 3M
```

Ranked approaches

1. **Naive: scan all 3M watches for this product, compare each to the new price in application code.** Correct, but $O(3M)$ of *application-level* work on every single price tick for this product — and this product ticks often, since it's popular. Slow, and it re-does the same comparison work for watchers who *already* matched on the previous tick.
2. **Sorted structure, range query (recommended baseline).** `ZRANGEBYSCORE alerts:iPhone16 899 +inf` returns exactly the matching set in one call, $O(\log n + k)$ — no wasted comparisons on the ~1.5M watchers who *don't* match. This is the fix from §7, and it's necessary but not sufficient here: `k` itself can be 1.5M, and pushing 1.5M notifications is real work.
3. **Only-notify-on-crossing (avoid repeat spam — the correctness-critical piece).** Immediately after reading the matched set, `ZREM` those members from `alerts:iPhone16`. Next tick, if the price is still \$899 or drops further, those users are *already gone* from the set — they don't get re-matched, so they don't get re-notified. If the price later rises back above their target (say back to \$950 — a restock-driven bump), **re- ZADD** them so their alert re-arms for the next real drop. This turns "matches on every tick while price stays low" into "matches exactly once per crossing," which is both a performance win (nothing to re-process) and a product-correctness win (nobody gets the same email 200 times).
4. **Bucket thresholds into coarse price buckets (interval index) to cut fan-out granularity.** Instead of one sorted set of exact prices, maintain, say, buckets every \$10 — `alerts:iPhone16:890-900`, `:900-910`, etc. A price update only needs to wake the buckets it crossed, which is useful when you want to batch-process "everyone in this \$10 band" as one unit of work (e.g., one Kafka message per bucket referencing a bucket ID, rather than one per user) — trading a little precision/latency for coarser, cheaper fan-out messages. Worth it mainly when `k` is enormous and per-user messaging becomes the bottleneck, not the lookup.
5. **Paginate the fan-out itself, don't materialize 1.5M results at once.** Even with #2+#3, handing 1.5M userIds to a single worker in one shot is a bad idea — cursor through `ZRANGEBYSCORE` with `LIMIT / WITHSCORES` in batches (or use `ZSCAN` -style iteration), pushing each batch to the notification Kafka topic as its own set of messages. This bounds memory in the matching worker

and lets notification throughput (email/push provider rate limits) be the thing that paces delivery, not the matching step.

Recommended approach

Lead with **#2 (sorted-set range query)** as the lookup mechanism — it's what makes "find exactly who crossed" cheap regardless of watcher count. Pair it, non-negotiably, with **#3 (remove-on-fire, re-arm-on-rise)** — without it, a product parked at a low price re-notifies its entire matched set on every single poll, which is both wasteful and a bad user experience. Add **#5 (paginated fan-out)** as soon as a single match set can be in the hundreds of thousands, so no single worker call ever has to hold millions of IDs in memory at once. Reach for **#4 (bucketing)** only if profiling shows the *notification fan-out volume itself* — not the lookup — is the bottleneck on the hottest handful of products.

What to say out loud: *"The sorted-set range query turns 'who just crossed their threshold' into a lookup that costs $O(\text{watchers of this product})$, not $O(\text{all watches in the system})$ — that's the win from Deep-dive A applied concretely. On top of that, removing matched watchers immediately and re-arming only when the price rises back above target is what prevents the same 1.5 million people from being re-notified on every subsequent tick while the price sits still. And once the matched set itself is huge, I paginate the fan-out so no single worker ever holds millions of IDs in memory — the lookup being cheap doesn't mean the delivery is free."*

Summary — the mental model

A price tracker is **not one hard problem, it's two, and they compound**: (1) a **fixed crawl budget** spread across a catalog that's orders of magnitude too large to poll uniformly at high frequency — solved by **priority-tiered scheduling** that spends the budget on popular/ volatile/actively-watched products, and (2) an **alert-matching operation whose cost must scale with the popularity of the product that changed, never with the size of the whole system** — solved by **indexing watches by product and keeping thresholds in a sorted structure** so a price tick becomes a targeted range query. Layer on a **time-series store** for price history (partitioned by product, because that's the access pattern), **async Kafka-backed notification fan-out** with remove-on-fire/re-arm-on-rise semantics so nobody gets spammed, and the system scales to millions of products and alerts without ever paying a cost proportional to the size of the whole catalog for a single price change.